

异常类和异常的处理机制

核心思想：**异常 是程序的一部分**

异常类继承顺序：

```
Throwable类 -> Error类      # 一般Error发生的情况，是无法通过软件解决的）
      | -> Exception类    # 狭义上的异常父类）#注意！Exception类也是一个受检异常#
      | -> RuntimeException # (运行时异常\非检查性异常\非受检异常)
      | -> 除了RuntimeException及其子类外，其他的异常都是 受检性异常\检查性异常
```

异常类型

受检查异常

指出代码可能会出现异常，并且要求开发者必须提供异常发生时的处理方法；

运行时异常（非受检异常

不强制检查问题的处理预案，写对了万事大吉，写错了执行就会崩溃；

比如：

1. 空指针异常
2. 数组下标越界异常
3. 类型转换异常
4. 算数运算异常

异常处理

抛出异常 throws/throw

通过throws，将异常抛给调用者

这里有两个关键字： throws 和 throw

throws： 声明方法抛出的异常的类型

```
// 例：
// 当 new FileInputStream("c:/1.txt"); 执行时
// "c:/1.txt"这个文件不存在，会抛出两个异常：FileNotFoundException, InterruptedException
// test函数使用throws关键字，告诉调用者，test函数可能会抛出这两个异常
public static void test() throws FileNotFoundException, InterruptedException
{
    FileInputStream fis = new FileInputStream("c:/1.txt");
    Thread.sleep(1000);
}
```

throw： 人造异常，主动抛出

```
//-----
// 例：
// 有需求：当参与加法的两个参数中，存在负数时，抛出异常
// -----

// 首先，我们应当先继承Exception类，写一个NegtiveNumberException子类
public class NegtiveNumberException extends Exception
{}

// 然后就可以通过new生成一个异常对象
// 并且通过throw关键字，把这个异常对象抛出
public static int add(int a, int b) throws NegtiveNumberException
{
    if(a <= 0 || b <= 0)
        throw new NegtiveNumberException(); //人为制造了一个Exception（一定发生）
    return a + b;
}
```

```
}
```

如果要抛出非受检异常： 继承`RuntimeException`

如果要抛出受检异常： 继承`Exception`

处理异常 try catch finally

try ... catch ... finally

这三个关键字之间可以组合使用：

1. try...catch...
2. try...finally...
3. try...catch...finally...

凡是出现catch的地方，均可以出现多个catch

```
try
{
    // 执行可能出现异常的代码
    A;
    B;
    C;
}
catch()
{
    // 如果出现了问题，进入这里进行处理
    D;
}
finally
{
    // 无论如何都会被执行
    E;
}
```

```
// finally无论如何都会被执行的
// 即使在C中执行return，代码的实际执行顺序也会是：A->B->E->C
```

catch可以捕捉多个异常

```
// 例：
try {
    test2();
} catch (InterruptedException e) {
    //处理线程中断异常
    e.printStackTrace();
} catch (FileNotFoundException e) {
    //文件未找到异常
    e.printStackTrace();
} catch (ParseException e) {
    //格式转换异常
    e.printStackTrace();
}
```

当多个异常的处理方式一样的时候，也可以一次性接受&处理

```
// 例：
// 如果多个异常的处理预案一致，也可以合并
try{
    test2();
} catch (InterruptedException | FileNotFoundException | ParseException e){
    //多个异常使用同一种处理预案
    e.printStackTrace();
}
```

如果不清楚异常的类型，还可以直接使用父类进行兜底

```
// 例：
// 不确定异常的类型和情况（通过Exception处理分支进行兜底）

try{
    test2();
}catch (Exception e){
    //处理所有异常
    e.printStackTrace();
}
```

Object类以及实例的五个操作

1. 业务逻辑相等

业务逻辑”相等“

Object类的equals方法默认等同于 ==

重写equals方法来实现

注意！这里说的是“重写” equals方法，重写的equals(Object o)的参数，也必须是Object o，否则就不是重写，而是重载了

```
public class Student
{
    private int sno;
    private String sname;

    @Override
    public boolean equals(Object obj) {
        if(obj instanceof Student){
            Student temp = (Student) obj;
            return (this.sno == temp.sno && this.sname.equals(temp.sname));
        }
    }
}
```

```
        return false;
    }
}
```

== 只能用于比较基本类型数据 是否一致

如果要比较引用类型数据是否一致，必须使用equals方法进行比较，这里尤其需要注意的是，比较String需要用equals

2. 字符串类型转换

重写Object类的toString方法，改变对象输出格式

```
public class Student
{
    private int sno;
    private String sname;

    @Override
    public String toString() {
        return "{sno: "+sno+", sname: "+sname+"}";
    }
}
```

3. Hash Code

Object有hashCode()方法

通过对象的“内存地址”计算一个哈希码；

特点：

1. 两个不同的hashCode代表的一定是两个不同的内存地址
2. 两个不同的内存地址，有可能会得到两个相同的hashCode

hashCode的顺序反映了内存里的位置顺序，所以可以使用hashCode让寻址更快

hashCode()方法可以被重写

```
public class Student
{
    private int sno;
    private String sname;

    @Override
    public int hashCode() {
        return Objects.hash(sno, sname); //利用sno和sname计算新的hashCode值
    }
}
```

4. 实例克隆

步骤：

1. 类需要实现Cloneable接口表示自身支持克隆
2. 重写Object类的clone方法（protected -> public） 直接使用super.clone() 即可实现

与 boolean equals(Object o) 类似；实现克隆接口的时候，也要注意，clone的返回值类型是 Object

```
public class Student implements Cloneable {

    private int sno;
    private String sname;

    @Override
    public Object clone() {
        try {
            Object o = super.clone();
            return o;
        }
    }
}
```

```
        } catch (CloneNotSupportedException e) {  
            throw new RuntimeException(e);  
        }  
    }  
}
```

5. 实例的可比较性

步骤：

1. 实现java.lang.Comparable接口，并重写其中的compareTo方法

原则：

1. compareTo()方法需要传入一个对象
2. 比较：比较的是传入的对象和this
3. 如果this在逻辑上比参数小，返回负数
4. 如果this在逻辑上和参数相等，返回0
5. 如果this在逻辑上比参数大，返回正数

```
public class Employee implements Comparable<Employee> {  
  
    public int empno;  
    public String ename;  
    public int salary;  
  
    public Employee(){}  
  
    @Override  
    public int compareTo(Employee o) {  
        //利用员工编号empno作为默认的比较规则  
        if(this.empno < o.empno){  
            //this的工号小于o的工号，认为this“小于”o，返回负数  
            return -1;  
        }else if(this.empno > o.empno){
```


